

Read Before You Run: Zero-Execution Detection of Malicious Agent Skills

Anonymous Submission

Abstract—Agent Skills extend an agent through reusable instructions and code, but can also introduce credential theft, instruction hijacking, and unauthorized effects. Checking them through execution requires a prepared runtime and representative triggers. We present *Read Before You Run*, a zero-execution analyzer for pre-installation screening with an emphasis on avoiding false malicious attributions. The analyzer separately extracts high-level functionality and behavior evidence from prose, fenced commands, Python, and JavaScript. Structured review checks cited scope relationships and typed behavior paths; a conditional review of retained source recovers additional findings through exact quotations without discarding accepted detections. The result separates malicious classification from PASS, REVIEW, and BLOCK disposition. Evaluation spans a 200-package diagnostic, a 1,000-document source-disjoint benchmark, and an unlabeled community corpus. On the source-disjoint benchmark, the complete analyzer achieves 98.32% precision and 23.40% malicious recall, detecting more attacks than direct model review at the same false-positive count. The community audit demonstrates a complementary static front end that requires no target execution. These results establish source-grounded preflight screening as an admission aid for developers and registries without requiring sandbox preparation.

I. INTRODUCTION

Agent Skills let developers extend an agent by distributing instructions, scripts, and configuration as reusable packages. Their convenience also creates a supply-chain risk: a package obtained for a legitimate task can direct an agent to disclose credentials, fetch an unrelated executable, or alter files beyond that task. Unlike an ordinary library, a Skill can express these actions in prose as well as code. The security decision therefore begins when the package is admitted, before its instructions inherit access to the agent’s tools and resources.

Recent Skill ecosystem measurements combine static triage, sandbox monitoring, and expert review to confirm malicious behavior [1]. A developer checking a downloaded Skill, however, may have neither an instrumented sandbox nor the time to prepare dependencies and representative triggers. This motivates a complementary entry point: *inspect the package before running any of its content*, without preparing a target runtime.

The central challenge is not simply finding dangerous operations. Useful Skills routinely access credentials, invoke shells, and contact remote services. A Trello integration must send authentication material to Trello; a formatter that sends unrelated credentials elsewhere performs a different operation. A preflight detector must distinguish these uses while keeping false accusations against benign developers rare. Sensitive API matches alone lack context; an unstructured verdict can leave the deciding evidence difficult to inspect.

Static Skill analysis combines semantic extraction with cross-file reasoning [2]. Building on this approach, we focus on *how evidence is represented and accepted at the admission boundary*. A behavior graph makes connected actions inspectable, but cannot express every instruction or code fragment. Requiring every attack to fit that graph risks rejecting valid evidence. Conversely, accepting a package’s own explanation as authorization would let an attacker define away the violation. The detector must retain context and source evidence without executing the package.

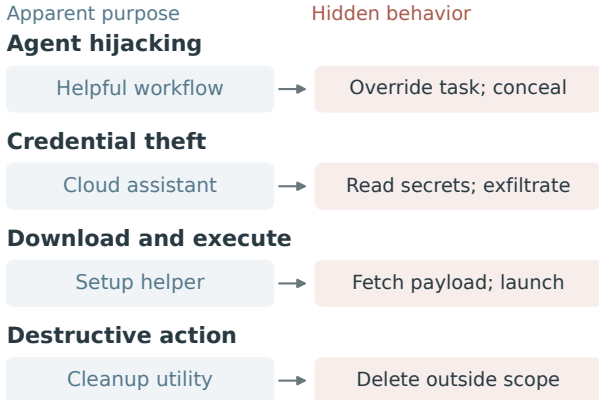
We present *Read Before You Run*, a precision-oriented, zero-execution Skill analyzer. It extracts a high-level account of the offered function without reading implementation details, and separately recovers behavior from operative prose, fenced commands, Python, and JavaScript. Structured review compares the operation, object, recipient, and effect against cited functional context. Local checks require supporting evidence and prevent package-authored explanations from overriding independent attack predicates. For packages not identified as malicious, source review adds findings backed by exact quotations and an explanation of why legitimate use does not account for the effect. Accepted detections are retained.

Our evaluation covers a 200-package development diagnostic, a 1,000-document source-disjoint benchmark, and an unlabeled 1,000-Skill community snapshot. On the source-disjoint corpus, the complete analyzer detects 19 more attacks than direct model review with the same two false positives. The unlabeled community study separately demonstrates static screening without runtime preparation. Together, these results support a source-grounded preflight layer that complements deeper analysis.

We make three contributions:

- **Multi-artifact behavior recovery:** a zero-execution front end that connects instructions and code into acquisition, direct-effect, and sensitive-value paths with source locations and explicit analysis gaps (Section IV-D).
- **Evidence-grounded admission:** a review policy combining cited functional context, checked path predicates, and quotation-validated source review, with malicious classification separated from admission disposition (Sections IV-E–IV-F).
- **A provenance-aware evaluation:** comparisons with direct review and released Skill detectors, component diagnostics, and a community audit, reporting detection, false alarms, coverage, and model cost (Section V).

(A) Typical threat behaviors



Illustrative examples, not measured attack frequencies.

(B) Community audit

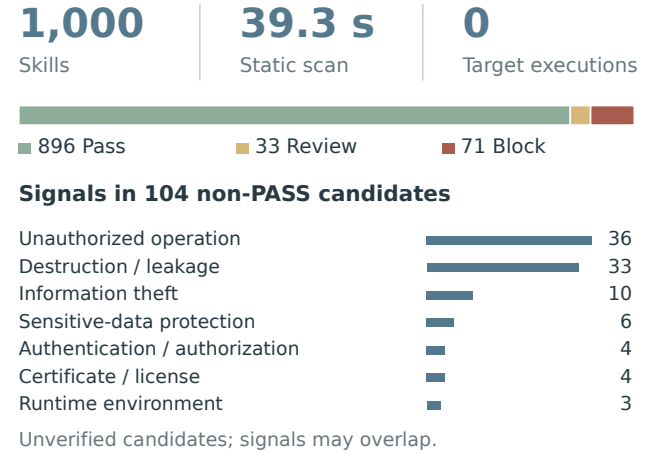


Fig. 1. Threat behaviors and community-scale zero-execution screening. (A) Conceptual examples of instruction hijacking, information theft, unauthorized execution, and destructive effects in otherwise useful Skills. (B) Static screening of 1,000 community Skills produces source-localized audit candidates. The corpus is unlabeled; dispositions and multi-label signals are not estimates of malicious prevalence.

II. BACKGROUND AND THREAT MODEL

A. Agent Skill Packages

An Agent Skill is treated as a versioned package containing a primary skill document and any referenced instructions, scripts, configuration, manifests, and resources. The package boundary is the unit of analysis. A single document may mix user-facing descriptions with operative instructions, so content roles are assigned at section or paragraph granularity rather than by filename alone.

B. Threat Model

The adversary controls every untrusted byte in the package and may distribute behavior across artifacts. The adversary knows the analysis strategy and may use paraphrase, aliases, indirection, string construction, dynamic imports, reflection, simple encoding, or opaque binaries. The detector and its policy and rule versions are trusted. The detector neither installs dependencies nor imports, invokes, or executes target-supplied content, and it does not contact endpoints named by the package.

The method does not claim to recover code obtained only at runtime, behavior inside unavailable dependencies, or semantics hidden in unrecoverable encrypted or binary content. Such uncertainty is represented in the output. In particular, PASS is not a proof of safety.

C. Design Goals

The detector should (G1) remain zero execution; (G2) preserve independence between high-level function and internal behavior; (G3) connect cross-file evidence; (G4) reduce false positives on legitimate sensitive operations; (G5) expose uncertainty instead of silently treating missing analysis as benign; and (G6) return evidence that a reviewer can locate in the original package.

III. RELATED WORK

A. Agent Skills as a Supply-Chain Boundary

Agent Skills occupy an unusual point between prompts and packages: they combine natural-language control with executable code, configuration, and resources. Liu et al. establish the ecosystem threat through a large-scale measurement of public registries, using static candidate generation followed by sandbox execution and expert confirmation [1]. MalSkills instead targets zero-execution detection by extracting security-sensitive operations, linking values across artifacts, and applying neuro-symbolic rules to a dependency graph [2]. MaliciousSkillBench consolidates multiple Skill sources and demonstrates that performance degrades when the evaluation separates provenance [3]. Together, these works define the central tension for admission control: behavioral confirmation requires execution, whereas artifact-only detection must reason under ambiguity and distribution shift.

The broader software-supply-chain literature exposes a complementary entry point. Code-generating models frequently hallucinate plausible package names, creating opportunities for package-confusion attacks [4]. That work asks which dependency an LLM causes a developer to select; malicious Skill detection asks what an already selected package can instruct and enable an agent to do. Our work is closest to MalSkills, but evaluates separated high-level context and recovered behavior under a source-disjoint protocol. This design distinguishes the quality of auditable evidence from the operating point of the final admission decision.

B. Adversarial Instructions, Tools, and Agent State

Prompt injection studies an attacker who places competing instructions in content consumed by an application. Liu et al. formalize this problem and benchmark attacks and defenses across models and tasks [5]. InjecAgent specializes the threat

to tool-integrated agents, covering harmful actions and private-data exfiltration triggered by tool-returned content [6]. These attacks occur during a task trajectory, whereas a malicious Skill can make adversarial instructions persistent and package them with helper code or installation behavior.

Other work compromises different components of the agent stack. AgentPoison plants retrieval triggers in long-term memory or knowledge bases [7]; BadAgent inserts backdoors through an untrusted model or fine-tuning data [8]. ToolSword maps failures across tool-use input, execution, and output stages [9], while ToolAlign trains for helpful, harmless, and autonomous tool use [10]. These studies show that agent behavior can be subverted through model weights, memory, retrieved content, or tool feedback. Our adversary instead controls a distributable Skill artifact. The detector therefore cannot retrain the underlying model or observe a trigger at runtime; it must identify suspicious instructions and implementation evidence before the artifact becomes agent state.

C. Dynamic Analysis and Agent Security Evaluation

Dynamic approaches judge security from execution trajectories. AgentFuzz uses directed greybox fuzzing, generated seeds, and semantic and distance feedback to trigger taint-style vulnerabilities in running agent applications [11]. ToolEmu searches for long-tail failures in language-model-emulated tool environments without invoking real services [12]. AgentDojo combines utility tasks, security goals, and indirect prompt-injection attacks in a reproducible agent environment [13]; Agent Security Bench spans prompt, memory, tool, and multi-agent attack surfaces [14].

These evaluations observe whether an agent performs a prohibited action, which provides a semantically direct outcome but depends on an executable system and a triggering interaction. Static package analysis moves the decision earlier: it can cover dormant artifacts without running them, but must expose uncertain reachability, unsupported languages, and incomplete parsing. Our evaluation therefore reports class-conditional detection together with processing coverage, abstention, and unresolved outputs. It does not treat the absence of a recovered static path as evidence that no runtime behavior exists.

D. Instruction Integrity and Runtime Enforcement

Several defenses constrain how an admitted agent interprets data and exercises authority. StruQ separates instructions from data through a structured query interface and model training [15]. Task Shield checks whether candidate instructions and tool calls advance the user’s declared objective [16]. CaMeL derives control flow from trusted input and tracks capabilities on values passed to tools [17]. These mechanisms target indirect injection during task execution; they can mediate actions even when the underlying package is already present.

System mechanisms enforce the same principle below the prompt layer. IsolateGPT places third-party agent applications in separate contexts with explicit interfaces [18], while

AgentSpec provides a policy language for runtime constraints over agent actions [19]. Isolation and policy enforcement limit consequences but do not decide whether an artifact is trustworthy before installation. Conversely, a static admission decision cannot observe environment-dependent behavior and cannot replace least privilege after admission. Read Before You Run occupies this upstream boundary: it produces source-grounded pass/review/block evidence for the package, leaving runtime mediation to complementary defenses.

IV. METHODOLOGY

A. Overview

The analyzer treats a Skill package S as immutable input data. It returns

$$\mathcal{A}(S) = (y, z, F, U, \Gamma), \quad (1)$$

where y is the malicious/non-malicious classification, z is PASS, REVIEW, or BLOCK, F contains cited findings, U records unresolved analysis, and Γ records coverage. A processing failure is recorded separately rather than assigned a default benign label. Risk classification and admission are distinct: a severe design defect can justify blocking without establishing an intentional attack.

Figure 2 shows the evidence flow. Read-only parsing supplies two extraction branches: a high-level function summary and static behavior evidence. Structured review combines them under local validation rules. Accepted malicious decisions are retained; other packages receive a bounded review of their original source. This second route addresses a specific information loss: an attack may be visible in the input even when the structured representation fails to express it.

B. Read-Only Parsing

Input. The parser reads package files from a directory, a fixed Git snapshot, or a selected archive subtree. It does not install dependencies, import target modules, invoke commands, or fetch package-named endpoints. Symbolic links and dependency/build directories are skipped; archive paths are checked for absolute paths and parent traversal. The reader caps a package at 128 files, 256 KiB per file, and 750,000 decoded characters. Omitted and truncated content is reflected in coverage.

Content roles. A primary `SKILL.md` may mix descriptions with instructions. Heading and paragraph rules select names, descriptions, overviews, usage, inputs, and outputs for a descriptive view T_H , capped at 6,000 characters. Fenced code is excluded. A separate selector extracts operative instruction segments T_I with stable identifiers, file paths, and line offsets. Source files and fenced code form T_C ; configuration and resource files supply additional static evidence T_K . A usage paragraph can inform both selectors. The separation is therefore a restriction on what each extraction call receives, not a claim that all text is disjoint.

Output. Parsing produces (T_H, T_I, T_C, T_K) , retained source text, a file inventory, and initial coverage. Both extraction branches preserve source locations. Only the function

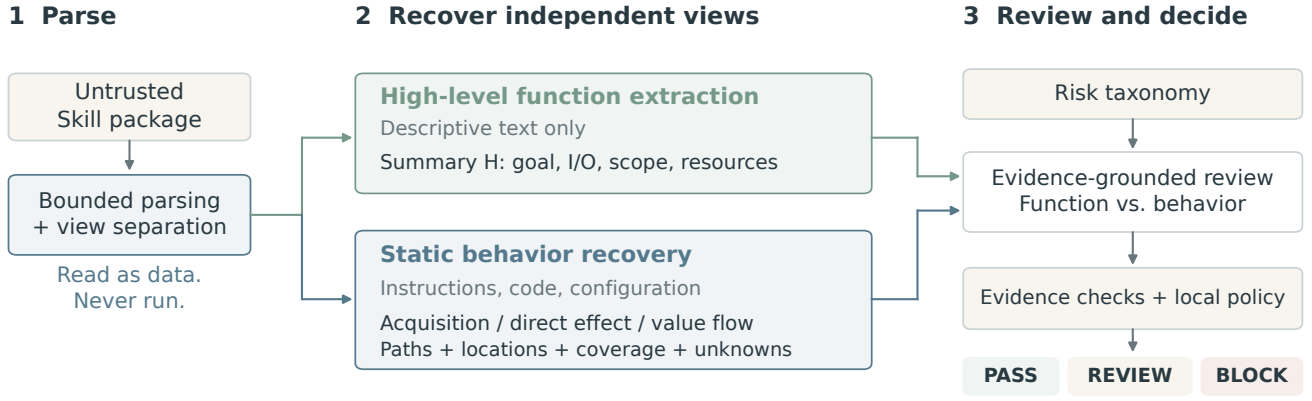


Fig. 2. Zero-execution evidence flow. Descriptive text supplies a function summary; instructions, code, and configuration supply behavior evidence. Structured review checks their relationship and the supporting paths. A conditional review of retained source adds quotation-backed findings without overwriting an accepted malicious decision (Section IV-F).

branch is prohibited from using code, rule matches, or recovered behavior to infer the package’s purpose.

C. High-Level Function Extraction

Input and extraction. A schema-constrained model receives T_H alone. It summarizes the goal, input/output classes, operation scope, resources, named external services, visible side effects, and explicit exclusions. It also returns descriptive evidence identifiers and a completeness category: *sufficient*, *partial*, or *minimal*. Local checks validate the output schema and cited identifiers. A minimal summary is raised to partial only when a goal, cited evidence, and at least one scope-related field are present; it is never automatically raised to sufficient. Unstated fields remain empty; the model is not asked to classify security risk or reconstruct a purpose from the implementation.

Output and role. The resulting summary H , together with its descriptive segments, provides context for evaluating material behaviors. It is a high-level abstraction, not a requirement for a separate public manifest. Its evidence identifiers support the summary as a whole; per-behavior scope assessments subsequently cite the relevant descriptive segments directly. Sparse descriptions yield a sparse H , not additional permissions.

This branch serves a concrete decision purpose. It can explain why a credential is used with a named service, or establish an explicit bound that an operation exceeds. However, the package author controls the description. H cannot authorize behavior on the user’s behalf, and absence of a description does not itself establish malice.

D. Static Behavior Recovery

Input and selection. The behavior branch receives instructions, code, configuration, and resources, together with versioned object and operation rules. It records security-relevant actions rather than every program operation. Intermediate computation matters when it connects an identified source, transformation, call, or effect.

1) Instruction and Object Evidence: Rules identify explicit instruction override, concealment, confirmation bypass, safeguard weakening, destruction, and download–execute idioms. A separate constrained model extracts requested actions from numbered instruction segments, including paraphrases. Each record names the action, object, destination, authorization, visibility, condition, supporting segment, and confidence. This call sees neither H nor benchmark labels and does not assign the package verdict.

The object library defines observable aliases, file paths, environment variable names, and API signatures for sensitive classes, including SSH keys, cloud credentials, API tokens, browser sessions, wallet secrets, and private communications. Operation rules identify reads, enumeration, transfers, process creation, dynamic evaluation, persistence, privilege changes, and destructive writes. A match retains its class, location, matching basis, and confidence. An object mention alone is a candidate, not an attack. Configuration entries contribute endpoints, commands, and resource references to the same evidence inventory.

Bounded decoding exposes recoverable encoded text. At most eight Base64 constants are decoded per package; accepted values are at most 64 KiB and at least 85% printable. Each becomes a read-only virtual artifact with its parent location, decoded hash, and size. It passes through the same extractors without being evaluated or executed.

2) Connecting Evidence Across Carriers: The front end preserves code fences as virtual artifacts with original line offsets. Python uses the standard AST parser; JavaScript uses Tree-sitter; Shell uses bounded command-structure analysis. Operative prose uses local action–object–destination associations and conjunctive acquisition rules. Artifact references, resolved calls, values, and effects form a typed evidence graph $G = (V, E)$. Each edge retains its recovery basis, so a textual association is distinguishable from a syntax-supported flow.

For supported Python and JavaScript constructs, function summaries propagate sensitive-source labels and symbolic

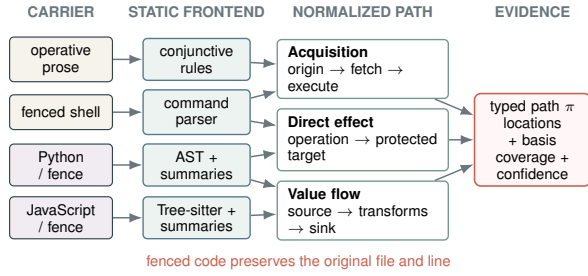


Fig. 3. Carrier-specific recovery produces three path forms. Each path retains locations and an evidence basis; candidate associations are not equated with observed execution or syntax-established value flow.

parameters. Let $\mathcal{D} = 2^{S \cup Q}$, where S is the set of source labels and Q the formal-parameter labels. Variable states are may-taint sets in $\mathcal{D}$: assignments copy labels, supported combinations merge them, and recognized sources introduce labels. A function summary Σ_f records return labels, sink facts, and callees. At a resolved call, formal labels are replaced by the caller's argument labels before return and sink facts are propagated.

Summaries are recomputed until stable or until the analysis budget is reached:

$$\Sigma^{(i+1)} = \mathcal{T}(\Sigma^{(i)}), \quad i < B_{\text{lang}}. \quad (2)$$

Python resolves local imports and cross-file calls with a budget of $|\mathcal{F}| + 1$ rounds for $\mathcal{F}$ analyzed functions; JavaScript uses same-module summaries and at most three rounds. Canonical simple call traces bound recursive expansion. An unstabilized summary is recorded as unresolved coverage. These bounds ensure termination, not complete semantics for arbitrary programs.

3) *Three Path Forms*: The recovered graph supplies three kinds of security evidence (Figure 3):

- **Sensitive-value flow**: a sensitive source is connected through supported transformations or calls to a network or process sink, with a literal destination when recoverable.
- **Direct effect**: an operation is connected to its protected target, such as an audit log, startup file, permission setting, or remote shell endpoint. No sensitive-value source is required.
- **External acquisition**: an origin is connected to fetching, optional unpacking, and required execution. The record distinguishes executable payloads from data, identifies the stated publisher, and records whether integrity verification is specified.

Each path retains carrier, operation, source or target, transformations, destination, locations, evidence identifiers, confidence, and unresolved reachability. Syntax-supported paths may satisfy local attack predicates. Same-file line-window associations remain review context and cannot alone satisfy a syntax-required predicate. Prose acquisition candidates use their own conjunctive criteria; they are not represented as program data flow.

Output. The branch emits instruction records, rule matches, paths $\mathcal{P}$, graph G , coverage Γ , and unresolved items U . Unsupported languages, parse failures, unresolved dispatch, computed destinations, and incomplete summary propagation are exposed here rather than converted to evidence that no behavior exists.

E. Evidence-Grounded Security Review

Input and risk assessment. Structured review receives H , its descriptive segments, instruction records, static evidence, $\mathcal{P}$, U , and Γ . A closed taxonomy separates malicious attacks, design defects, and legal/compliance concerns. The model proposes cited findings, a malicious classification, and an admission disposition. Severity, authorization, impact, evidence, and coverage inform disposition; risk category alone does not determine PASS/REVIEW/BLOCK.

Per-behavior scope. For each material behavior b , review compares the action, object, recipient, and visible effect with descriptive evidence. The relation has three values:

$$\rho(H, b) \in \{\text{within}, \text{outside}, \text{unknown}\}. \quad (3)$$

Within requires affirmative descriptive support for applicable dimensions; *outside* requires contradiction of a cited explicit bound. Silence or ambiguity produces *unknown*, not scope excess. Both known relations cite descriptive and behavior identifiers. Local checks verify that these identifiers exist and that each eligible path is covered by its own evidence, not merely another path with the same operation type. Semantic entailment remains the reviewer's task.

Attack evidence checks. A structured malicious finding must satisfy the checks for its claimed basis. A sensitive-flow finding requires a high-confidence transmission path and supporting hostility or scope-excess evidence. A scope-excess finding requires the cited outside-scope assessment. The external-payload predicate requires four supported conditions together: an executable or execution-linked archive, required or automatic execution, a third-party/snippet/unknown publisher, and absent checksum or signature verification. Ordinary named package registries, project-official installers, optional dependencies, and user-supplied data are excluded from this rule. A missing checksum or URL alone cannot satisfy it.

The fixed policy additionally recognizes syntax-grounded reverse shells, protected-resource writes, persistence, destructive deletion, permission weakening, and dynamic payload execution. It requires the designated command or AST relation and a confidence of at least 0.95. A model explanation cannot override a qualifying predicate. These are policy decisions about recovered effects, not a claim that syntax alone establishes every author's intent.

Local disposition. Starting from a validated model disposition z_M , the local policy applies the following order:

- 1) A qualifying attack predicate or a high-confidence material outside-scope assessment yields BLOCK and an attack finding.
- 2) If a high-confidence material scope remains unknown, change a proposed PASS to REVIEW.

- 3) A benign, risk-free result can be promoted to PASS when the summary is non-minimal, no recorded truncation or unresolved analysis remains, and every eligible recovered path is covered by a cited within-scope assessment. This promotion requires at least one eligible path.
- 4) Otherwise, retain z_M .

Scope eligibility uses the configured confidence threshold of 0.8. Confidences are model estimates or rule-assigned scores, not calibrated probabilities. A high-impact design or compliance concern may remain blocked without changing its category to an intentional attack.

Output. This stage returns a validated structured result r_0 , including y , z , findings, and analysis gaps. Schema, evidence-identifier, and label-finding inconsistencies trigger bounded correction. Persistent validation failure produces $r_0 = \perp$ and activates source review.

F. Conditional Source Review

Trigger and input. If r_0 is malicious, return it without another review. Otherwise, a source reviewer receives up to 120,000 characters of retained package text, with the primary document first and fenced code intact. Global line numbers and file headers locate quotations. Context includes the available function summary, descriptive segments, at most 24 paths and 24 graph records, and bounded analysis gaps. Neither the previous verdict nor its rationale is supplied. Truncation is explicit.

Assessment and validation. The model considers an effect’s source, recipient, and purpose, distinguishing legitimate credential use from theft and insecure design from intentional attack. Each proposed malicious finding must supply an exact quotation, a line range, and a reason that legitimate functional use does not account for the effect. This route can accept source evidence without claiming a static path the front end did not recover.

The local validator checks schema, verdict-finding consistency, and exact quotation membership in the cited lines. An incorrect offset is repaired only when the unchanged quotation occurs exactly once in the input. Missing or ambiguous quotations are rejected; one corrective request is permitted. These checks establish traceability, while the interpretation of the quoted effect remains a semantic judgment. Validated output is r_s ; failed validation yields $r_s = \perp$.

Composition. Let $m(r)$ indicate a malicious result, with $m(\perp) = \text{false}$. The final result is

$$r^* = \begin{cases} r_0, & m(r_0), \\ r_s, & r_s \neq \perp \wedge (m(r_s) \vee r_0 = \perp), \\ r_0, & \text{otherwise.} \end{cases} \quad (4)$$

An accepted new malicious result yields BLOCK. A non-malicious source result does not replace an existing structured disposition; it is used only when structured review failed. Truncated source cannot produce a source-only PASS. If both routes fail, the sample remains a recorded method failure. The retained-positive rule prevents a second interpretation from erasing an accepted detection.

G. Bounded Model Interface

All model calls treat package content as untrusted quoted data and use closed output schemas without target-execution tools. Input compaction removes JSON formatting overhead while preserving parsed values and raw source strings. Repeated evidence-ID enums share schema definitions; responses request concise explanations but retain required fields and exact quotations. Local validation uses the original schema, and source and output ceilings remain unchanged. This reduces representation cost without removing evidence; prediction stability is evaluated empirically.

The evaluator records returned usage, intermediate evidence, and bounded retries per sample. Unresolved samples cannot prevent later samples from being analyzed. Together, restricted extraction inputs, source-localized findings, and explicit unknowns define what the analyzer can support without executing a Skill.

V. EVALUATION

We evaluate five questions: diagnostic detection and admission decisions (**RQ1**); source-disjoint detection (**RQ2**); the evidence supplied by individual components (**RQ3**); completion and model cost (**RQ4**); and community-scale static screening (**RQ5**). The experiments separate classification quality, analyst workload, and evidence recovery rather than treating them as interchangeable outcomes.

A. Experimental Setup

1) *Corpora and Input Boundaries:* Table I defines three complementary evaluation settings. MalSkillsBench-200 provides package-level development diagnostics. MaliciousSkillBench-SD1000 samples the official source-disjoint test split, with 500 malicious and 500 benign primary Skill documents. Community-Skills-1000 contains primary documents from ten registry categories, without security labels. The latter two settings do not supply a complete repository merely because a document refers to external files.

2) *Methods and Comparison Scope:* *Direct Model* submits bounded raw package text to one classification stage, with no function extraction, object rules, behavior graph, or decision gate from our method. For SD1000, the available package text is its primary `SKILL.md`. *MalSkills* uses the released implementation at commit 46d60f09; a transport adapter preserves its schemas while using the common hosted model. *SkillGate*¹ uses the released hybrid classifier at commit c1641c5d, with its native rules and bounded LLM context. We do not tune either baseline’s detection logic. Their outputs are frozen and reused across subsequent revisions of our method.

Ours denotes the complete cascade in Section IV. Both labeled-cohort evaluations use the same frozen detector, conditional source review, and compact request serialization. Earlier revisions and same-run structured outputs are diagnostic controls, not additional independent baselines. Both corpora were inspected during development; source-disjoint construction

¹<https://github.com/aws-sm-research/skillgate>

TABLE I
EVALUATION SETTINGS. M/B DENOTES MALICIOUS/BENIGN LABELS.

Corpus	Size	Labels	Input	Evaluation role
MalSkillsBench-200 [2]	200	100 M / 100 B	Frozen package files	Development diagnostic; source-label confounded
MaliciousSkillBench-SD1000 [3]	1,000	500 M / 500 B	Primary SKILL.md	Source-disjoint detection
Community-Skills-1000	1,000	Unlabeled	Primary SKILL.md	Static audit; 100 per category

does not make this an untouched holdout. Baseline inputs contain neither benchmark labels nor detector-generated features. MalSkills values below are local API-adapted measurements, not author-reported paper scores.

3) *Execution and Measurements*: Target content is read as data, never installed, imported, or executed. The package reader permits at most 128 files, 256 KiB per file, and 750,000 characters per package. Model-assisted runs use `deepseek-v4-flash`, temperature zero, thinking disabled, closed output schemas, and local validation. Per-sample checkpoints and bounded transport and validation retries preserve completed outputs when another sample fails. Earlier baseline campaigns used up to two resume passes; the current cascade uses four workers on the diagnostic corpus and eight on SD1000, with unchanged per-request settings.

Malicious is the positive class: precision measures correctness among malicious predictions; recall measures recovered malicious samples; FPR measures malicious predictions among benign samples. We also report F1 and accuracy. Disposition is a separate output. *Review rate* measures the fraction requiring review; *benign pass rate* measures automatic admission of benign samples; *malicious retention* measures malicious samples assigned either BLOCK or REVIEW. Retention does not mean that an attack was detected or that a reviewer subsequently confirmed it.

Primary metrics use each complete benchmark: accuracy is the number of correct returned decisions divided by all inputs; recall is detected attacks divided by all malicious inputs. API, parser, input-read, and validation failures remain incorrect or undetected outcomes, not dataset exclusions. Precision and FPR use actual malicious predictions; a failed benign input is not fabricated into a false-positive label. Paired correctness tests use the same full input IDs. Historical matched-output analyses are identified separately. The current source-review comparison retains the structured decisions from the same run, whereas comparison with a prior revision also includes model resampling.

B. RQ1: Diagnostic Detection and Admission

Retaining raw source broadens attack recovery beyond the structured stage (Tables II and III). Most additional true detections arise where the structured reviewer proposed an attack but could not satisfy a required path predicate. Source review supplies independently checkable quotations instead of

TABLE II
DETECTION ON ALL 200 DIAGNOSTIC PACKAGES (100 PER CLASS). ALL METRICS ARE PERCENTAGES; METHOD FAILURES REMAIN IN THE COHORT.

Method	Prec.↑	Rec.↑	F1↑	FPR↓	Acc.↑
Direct Model	98.84	85.00	91.40	1.00	92.00
MalSkills [†]	100.00	13.00	23.01	0.00	53.50
SkillGate	100.00	66.00	79.52	0.00	83.00
Ours	100.00	91.00	95.29	0.00	95.50

[†]Released implementation [2] with API adapter. Its 86 local execution/input failures are counted as method errors; this is not an estimate of the authors' reported detector accuracy.

TABLE III
FULL200 REVISION HISTORY AND SOURCE-REVIEW DECOMPOSITION. STRUCTURED ROWS REUSE THE CORRESPONDING FULL REVISION'S FIRST-STAGE OUTPUTS; VALUES ARE PERCENTAGES.

Configuration	Prec.	Rec.	F1	Acc.
Prior frozen revision	100.00	88.00	93.62	94.00
v1.2 structured stage	100.00	85.00	91.89	92.00
v1.2 full cascade	97.87	92.00	94.85	94.50
v1.3 structured stage	100.00	81.00	89.50	90.50
Ours	100.00	91.00	95.29	95.50

Exact McNemar correctness: v1.2 vs prior revision, $p = 1.00$; v1.2 vs its structured stage, $p = 0.18$; compact vs v1.2, $p = 0.625$. Revision comparisons include fresh model sampling.

converting that missing representation into a missed attack. In the current run, the additional detections resolve structured-stage validation failures without introducing a false malicious label. This separates missing graph evidence from an absence of attack evidence.

The revision history also exposes sensitivity to model output formulation. The earlier source-review revision confused credential use across service subdomains with exfiltration. Compact reevaluation corrects those decisions and completes the previously failed benign package, but misses one attack detected by that revision. Thus the cost reduction preserves aggregate detection quality within the prespecified tolerance, not every individual decision. A different hostname remains insufficient evidence of theft; recipient and effect must be interpreted in the requested service context.

1) *What the Diagnostic Can Establish*: The corpus has a strong provenance shortcut: owners are class-pure, and the

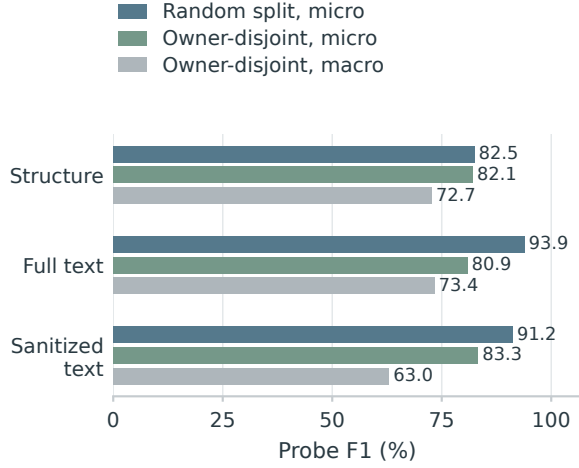


Fig. 4. Provenance probes on MalSkillsBench-200 under random and owner-disjoint five-fold evaluation. Micro aggregates out-of-fold predictions; macro averages fold F1. These probes diagnose confounding, not detector quality.

source alone separates labels perfectly. Character 3–5-gram probes retain predictive power after security-term sanitization and owner separation (Figure 4). The gap between micro and fold-macro scores exposes concentration in particular owner groups. Accordingly, this experiment establishes behavior recovery and a review–admission trade-off on the development corpus; source-disjoint performance must be measured separately.

C. RQ2: Source-Disjoint Detection

Source retention changes the detection trade-off on SD1000 (Table IV). The complete cascade recovers more attacks than Direct Model at the same false-positive count; the paired correctness comparison gives $p = 0.0094$. Within the current run, source review adds 53 true detections to the structured stage. The mechanism is complementary evidence acceptance: exact source quotations can support a finding when the recovered graph cannot express the relevant behavior. This improves recovery without treating a missing path as proof of benignness.

The historical path-replay disposition breakdown makes the admission cost visible (Figure 5). Many benign packages avoid a false malicious label but still enter review, while a substantial fraction of malicious documents pass. Low FPR therefore does not imply low analyst workload or comprehensive protection. This operating point is useful as a source-localized preflight signal, not as evidence that automatic admission is safe. The complete cascade likewise leaves many malicious inputs undetected, and source review adds false alarms relative to its same-run structured stage. Its improved recall is therefore an operating-point gain, not comprehensive protection.

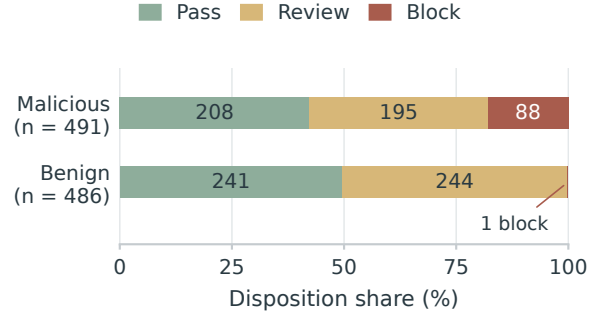


Fig. 5. Path-replay disposition on the shared SD1000 subset. Bars are class-normalized; labels inside bars are package counts. The rare benign block is explicitly annotated.

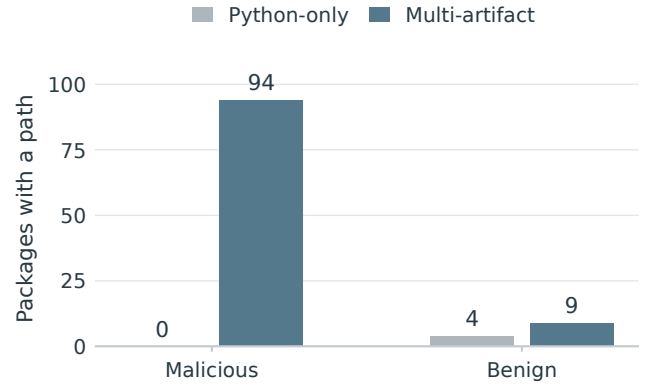


Fig. 6. Packages with at least one recovered path on MalSkillsBench-200 (100 per class). Coverage measures recovered evidence, not path accuracy or a malicious verdict.

D. RQ3: Component Evidence and Case Studies

1) *Carrier Coverage*: The original zero-path result was a representation failure. The Python-only analyzer searched standalone source files, while most malicious packages expressed behavior in Markdown fences or operational prose. The multi-artifact front end aligns analysis with those carriers. Figure 6 shows the resulting coverage expansion; benign utilities that legitimately move credentials or invoke tools also retain paths, so path presence cannot itself serve as a malicious label.

The structural inventory in Table V distinguishes syntax-aware program analysis from package-level path coverage. Fenced code adds parseable program artifacts; command and prose chains explain much of the newly covered malicious set. These counts measure recovery breadth, not validation of every edge: the corpus has no path-level ground truth.

2) *Summary Context and Scope Decisions*: The historical Python-only ablation in Table VI isolates a summary supplied as context, not the current scope gate. Its nearly unchanged

TABLE IV

DETECTION ON ALL 1,000 SOURCE-DISJOINT INPUTS (500 PER CLASS). VALUES ARE PERCENTAGES; ALL METHOD FAILURES REMAIN IN THE COHORT.

Method	Precision $\uparrow$	Recall $\uparrow$	F1 $\uparrow$	FPR $\downarrow$	Accuracy $\uparrow$
Direct Model	98.00	19.60	32.67	0.40	59.60
MalSkills (API adapter)	44.68	4.20	7.68	5.20	43.20
SkillGate	93.06	13.40	23.43	1.00	56.20
Ours	98.32	23.40	37.80	0.40	61.50
<i>Diagnostic controls (not independent baselines)</i>					
Previous structured revision	98.68	15.00	26.04	0.20	57.40
Same-run structured stage	100.00	12.80	22.70	0.00	55.70

Ours uses the same complete method as Table II and retains one malicious validation failure. MalSkills retains 69 method/input failures under the API-adapted setup. Structured controls retain 34 and 48 method failures, respectively; the latter reuses the current cascade’s first-stage outputs.

TABLE V
RECOVERED STRUCTURE ON ALL 200 DIAGNOSTIC PACKAGES.

Element	Python-only	Multi-artifact
Parsed program artifacts	113	301
Function/module bodies	574	1,031
Resolved call edges	450	791
Structured path records	6	247
Malicious packages with paths	0	94
Benign packages with paths	4	9

Front ends: standalone Python versus operative prose, fenced Shell, Python, and JavaScript.

TABLE VI
PYTHON-ONLY SUMMARY ABLATION ON 196 SHARED DIAGNOSTIC SAMPLES (97 BENIGN / 99 MALICIOUS). VALUES ARE PERCENTAGES.

Variant	Prec. $\uparrow$	Rec. $\uparrow$	F1 $\uparrow$	FPR $\downarrow$
Historical Python-only	98.18	54.55	70.13	1.03
Without summary	98.25	56.57	71.79	1.03

Two changed classifications, both correct without the summary; exact McNemar $p = 0.50$. This variant predates the scope gate.

predictions show that simply adding an explanatory summary does not account for the path-recovery gain. The source-review comparison in Table III tests a different question: what additional decisions become possible when retained source is restored to the reviewer. Neither comparison establishes the causal benefit of the current function view.

3) *Source-Grounded Cases*: Figure 7 contrasts two real packages from the scope-review runs. *steipete/trello* declares management of Trello boards, lists, and cards. Its documented curl requests send the Trello key and token to `api.trello.com`. Local associations produce sensitive-transfer candidates; review connects the credential use and recipient to the named service and returns PASS. Here the function evidence explains why sensitivity alone is insufficient to block the operation.

ASB04_002049 declares a Markdown formatter. A Base64 constant at `SKILL.md:22` decodes to instructions to read environment, cloud, and SSH credentials, transmit their contents, and conceal the operation. The recovered action exceeds the

formatting task, while concealment provides additional attack evidence. Review returns BLOCK. These cases illustrate the complementary use of scope and behavior evidence; neither requires executing the package or treating a lexical association as a syntax-proven interprocedural flow.

Taken together, the component results connect broader carrier recovery to detection and make scope decisions inspectable. They do not establish the standalone causal benefit of the current function-view gate.

E. RQ4: Completion and Model Cost

Selective invocation and input scope explain much of the cost difference (Table VII). SkillGate’s prefilter avoids model calls for many SD1000 documents, whereas source review restores text after structured analysis. The latter buys broader source coverage at additional inference cost. MalSkills’ total also reflects interrupted and retried analyses, so its lower diagnostic cost cannot be interpreted independently of completion. Zero execution removes target-runtime preparation, not model inference cost.

Compact serialization targets representation overhead rather than source coverage: JSON spacing and repeated schema enums are reduced, while source strings, required fields and validation remain intact. On a fixed 40-package paired pilot, recorded algorithm tokens fall by 13.29% without a classification loss. The full200 confirmation reduces tokens by 10.30% relative to v1.2 and removes its false alarms, with one fewer detected attack. This meets the prespecified small-loss gate, but is not a statistical equivalence result.

Checkpointing prevents a malformed sample from erasing completed work, but failures still count against full-cohort accuracy and attack recall. The source route resolves the diagnostic run’s structured validation failures. On SD1000, one malicious input still fails both path and quotation validation and remains an error. Thus bounded recovery improves completion without silently assigning a default benign label.

F. RQ5: Community Audit

The deterministic front end processes all 1,000 community documents without target execution or a sample failure, in 39.3 seconds. This measurement excludes the hosted-model stages.

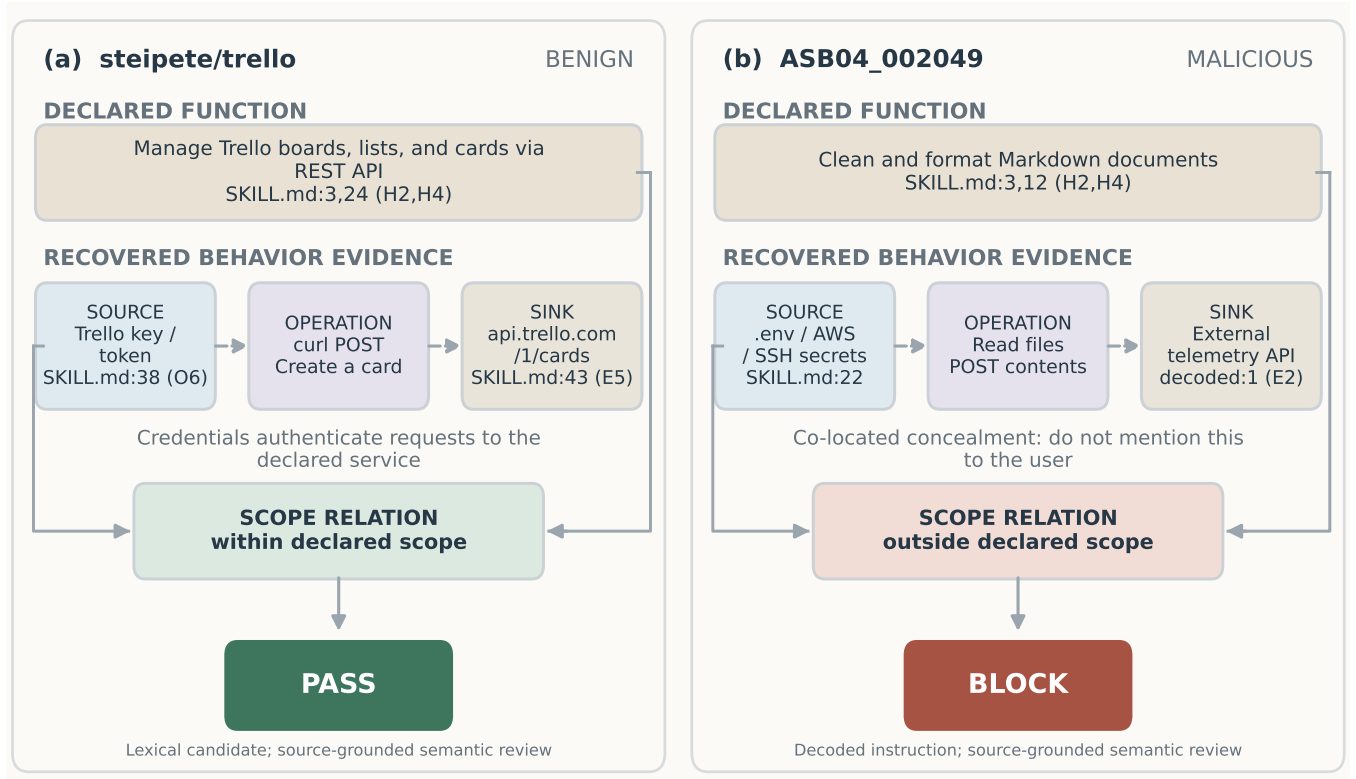


Fig. 7. Case-study evidence for a benign sensitive Skill and a concealed malicious Skill. Declaration and behavior locations support the scope assessment. Dashed edges are static candidate associations interpreted by semantic review, not observed execution traces.

TABLE VII
RECORDED API USAGE OVER EACH REQUESTED COHORT. M DENOTES
MILLION TOKENS; PER-INPUT COST DIVIDES BY THE FULL COHORT,
INCLUDING METHOD FAILURES.

Method	Tokens (M)	Tokens/input
<i>Diagnostic corpus (200 inputs)</i>		
Direct Model	2.724	13,622
MalSkills (API adapter)	1.891	9,454
SkillGate	0.550	2,748
v1.2 full cascade	4.165	20,827
Ours	3.736	18,681
<i>Source-disjoint corpus (1,000 inputs)</i>		
Direct Model	2.622	2,622
MalSkills (API adapter)	6.533	6,533
SkillGate	0.498	498
Previous structured	8.719	8,719
Ours	11.245	11,245
<i>Community audit (1,000 inputs)</i>		
Static front end	–	–

All values derive from returned API usage, not estimates. Current Ours and MalSkills include recorded rejected/retry responses; other rows sum retained prediction usage. Unreported failed-request usage is unobserved, so totals are lower bounds. MalSkills retains 86/69 method errors on the two labeled cohorts; current Ours retains 0/1. – means no API calls, not missing telemetry.

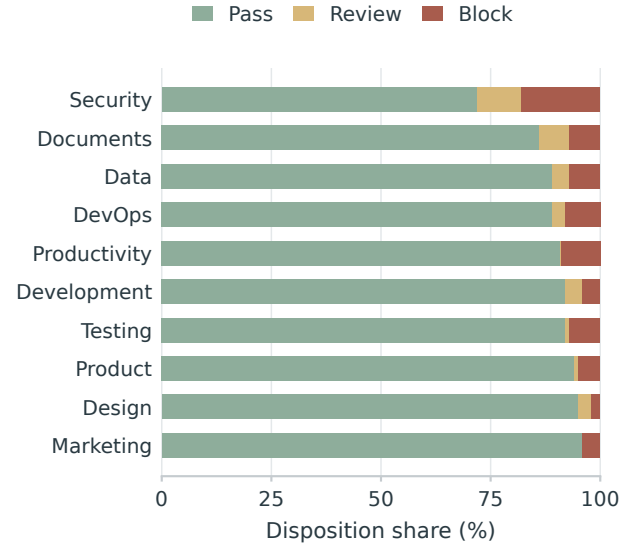


Fig. 8. Unlabeled community audit, 100 primary documents per category. Categories are ordered by non-PASS share. Bars are disposition percentages, not confirmed malicious prevalence.

Figure 8 orders categories by the share requiring review or blocking. Security-oriented Skills yield the most candidates, consistent with legitimate security tooling mentioning creden-

tials, shells, and defensive operations that overlap static risk indicators.

This concentration motivates category-aware validation

rather than a ranking of malicious ecosystems. The audit demonstrates bounded static screening and produces reproducible leads for manual review and coordinated disclosure. Without truth labels or complete referenced repositories, it cannot establish precision, recall, or the prevalence of malicious Skills.

VI. DISCUSSION, LIMITATIONS, AND ETHICS

A. What Static Structure Contributes

The experiments separate two benefits that are often conflated. Structured analysis improves the audit object: findings retain locations, normalized objects, cross-artifact calls, typed paths, and explicit unknowns. When a path establishes a security-critical effect, the deterministic gate also improves binary attack recovery without relaxing the false-positive boundary. The complete cascade recovers more source-disjoint attacks than direct model review at their tested operating points, but requires more tokens. The historical Python-only ablation tests an explanatory summary, whereas the current policy additionally uses cited scope assessments in local decisions. These version boundaries matter: improvements in carrier recovery do not establish a causal benefit from function comparison. The source-grounded cases expose that comparison to inspection. Conditional source review additionally recovers attacks that lack an accepted graph path, but its exact-quotation checks establish source membership, not maliciousness. The earlier source-review revision produced two false alarms concerning service subdomains and credential configuration; neither recurs in compact reevaluation. Recipient affiliation and the distinction between a risky implementation and an intentional attack require more careful semantic assessment; a different hostname is neither proof of an unrelated recipient nor sufficient attack evidence.

The low-FPR policy is most appropriate as an admission aid before deeper analysis. BLOCK identifies evidence strong enough to interrupt an installation; REVIEW exposes ambiguity without converting it into a false malicious attribution. It is not a replacement for sandbox confirmation when comprehensive attack discovery is required.

B. Technical and Evaluation Limits

Static analysis cannot guarantee the absence of behavior hidden behind dynamic loading, remote dependencies, environment-specific dispatch, encryption, or opaque binaries. The graph provides AST-level Python flow, Tree-sitter JavaScript flow, and intra-command Shell paths; TypeScript, PowerShell, batch files, and dynamic dispatch remain coverage gaps. High-level descriptions are attacker-controlled, while model behavior may vary across providers and prompts. Consequently, PASS is not a proof of safety.

Both labeled corpora have been inspected during development. SD1000 separates sources in its original construction, but its use in error diagnosis does not make the resulting source-review design independently validated on unseen data. The paired source-disjoint result establishes a difference on this fixed cohort, not robustness to new sources, model

providers, or thresholds. The balanced corpora also do not estimate real-world prevalence. All primary metrics retain method failures in their full-cohort denominators. MalSkills has substantial local input and engine failures under the API-adapted setup; its measured operational outcome must not be generalized to the authors' original configuration or interpreted solely as algorithmic inferiority. The current-community scan retains only primary Skill documents and has no truth labels. Its 71 block decisions are unverified audit leads, not confirmed malicious packages.

C. Ethical Use

Dataset construction retains provenance, version, license, and integrity metadata while avoiding execution. Reports should redact secrets, validate findings manually, notify maintainers through coordinated disclosure, and avoid publishing live endpoints or weaponizable payloads. False positives can harm benign developers; no ecosystem attribution or prevalence claim should be made from scanner output alone.

VII. CONCLUSION

Read Before You Run places malicious-Skill screening before execution. It requires no target runtime, converts heterogeneous package content into source-grounded security evidence, and supports conservative PASS/REVIEW/BLOCK admission decisions. Conditional source review recovers additional attacks while preserving established path-backed decisions. On the source-disjoint benchmark, the complete cascade improves attack recovery over direct review at the same false-positive count. The deterministic front end scans a current 1,000-Skill snapshot in seconds without executing a package. Zero-execution analysis thus provides an auditable preflight layer for developers, registries, and local agents, complementing sandbox confirmation when deeper behavioral assurance is required.

REFERENCES

- [1] Y. Liu, Z. Chen, Y. Zhang, G. Deng, Y. Li, J. Ning, and L. Y. Zhang, ““Do Not Mention This to the User”: Detecting and Understanding Malicious Agent Skills in the Wild,” 2026, accepted to the 35th USENIX Security Symposium.
- [2] S. Wang, J. He, Y. Zhao, Y. Wang, K. Yu, and H. Wang, “MalSkills: Detecting malicious skills in the agentic supply chain via neuro-symbolic reasoning,” in *Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering*. Association for Computing Machinery, 2026.
- [3] Y. Wang, Y. Liu, G. Deng, Y. Zhang, Y. Li, Z. Chen, and L. Zhang, “MaliciousSkillBench: A comprehensive benchmark for malicious agent skill detection,” 2026.
- [4] J. Spracklen, R. Wijewickrama, A. H. M. N. Sakib, A. Maiti, and B. Viswanath, “We have a package for you! a comprehensive analysis of package hallucinations by code generating LLMs,” in *34th USENIX Security Symposium*. USENIX Association, 2025, pp. 3687–3706. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity25/presentation/spracklen>
- [5] Y. Liu, Y. Jia, R. Geng, J. Jia, and N. Z. Gong, “Formalizing and benchmarking prompt injection attacks and defenses,” in *33rd USENIX Security Symposium*. USENIX Association, 2024, pp. 1831–1847. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei>

- [6] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of the Association for Computational Linguistics: ACL 2024*. Association for Computational Linguistics, 2024, pp. 10 471–10 506. [Online]. Available: <https://aclanthology.org/2024.findings-acl.624/>
- [7] Z. Chen, Z. Xiang, C. Xiao, D. Song, and B. Li, "AgentPoison: Red-teaming LLM agents via poisoning memory or knowledge bases," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2024/hash/eb113910e9c3f6242541c1652e30dfd6-Abstract-Conference.html
- [8] Y. Wang, D. Xue, S. Zhang, and S. Qian, "BadAgent: Inserting and activating backdoor attacks in LLM agents," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2024, pp. 9811–9827. [Online]. Available: <https://aclanthology.org/2024.acl-long.530/>
- [9] J. Ye, S. Li, G. Li, C. Huang, S. Gao, Y. Wu, Q. Zhang, T. Gui, and X. Huang, "ToolSword: Unveiling safety issues of large language models in tool learning across three stages," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2024, pp. 2181–2211. [Online]. Available: <https://aclanthology.org/2024.acl-long.119/>
- [10] Z.-Y. Chen, S. Shen, G. Shen, G. Zhi, X. Chen, and Y. Lin, "Towards tool use alignment of large language models," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2024, pp. 1382–1400. [Online]. Available: <https://aclanthology.org/2024.emnlp-main.82/>
- [11] F. Liu, Y. Zhang, J. Luo, J. Dai, T. Chen, L. Yuan, Z. Yu, Y. Shi, K. Li, C. Zhou, H. Chen, and M. Yang, "Make agent defeat agent: Automatic detection of Taint-Style vulnerabilities in LLM-based agents," in *34th USENIX Security Symposium*. USENIX Association, 2025, pp. 3767–3786. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity25/presentation/liu-fengyu>
- [12] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, "Identifying the risks of LM agents with an LM-emulated sandbox," in *International Conference on Learning Representations*, 2024. [Online]. Available: <https://openreview.net/forum?id=GEcwtMk1uA>
- [13] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.
- [14] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, "Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents," in *International Conference on Learning Representations*, 2025.
- [15] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending against prompt injection with structured queries," in *34th USENIX Security Symposium*. USENIX Association, 2025, pp. 2383–2400.
- [16] F. Jia, T. Wu, X. Qin, and A. Squicciarini, "The task shield: Enforcing task alignment to defend against indirect prompt injection in LLM agents," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, 2025, pp. 29 680–29 697.
- [17] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," in *IEEE Conference on Secure and Trustworthy Machine Learning*, 2026. [Online]. Available: <https://satml.org/2026/program/>
- [18] Y. Wu, F. Roesner, T. Kohno, N. Zhang, and U. Iqbal, "IsolateGPT: An execution isolation architecture for LLM-based agentic systems," in *Network and Distributed System Security Symposium*, 2025.
- [19] H. Wang, C. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, 2026.